

Confronto di Metodi di Ottimizzazione per la Classificazione Binaria

Lorenzo Chesta

- Dataset
- Preprocessing
- Modello
- Loss function
- Gradient Descent, SGD, Newton
- Considerazioni finali

Dataset

Breast Cancer Wisconsin (Diagnostic) Dataset:

- rappresenta un **problema reale**
 - ogni campione è un prelievo di agoaspirato classificato come 'Malignant' o 'Benign'
 - 212 'M', 357 'B'
- dataset **semplice**
 - le differenze nei risultati sono attribuite principalmente ai metodi di ottimizzazione piuttosto che alla complessità del modello

Preprocessing

Standardizzazione:

$$z = \frac{x - \mu}{\sigma}$$

- x : valore della feature
- μ : media dei valori della feature
- σ : deviazione standard dei valori della feature
- z : valore della feature standardizzato (media 0, deviazione standard 1)

Preprocessing

Standardizzazione:

- problema **ben condizionato**
 - riduce le oscillazioni e accelera la convergenza dei metodi basati sul gradiente
- porta le feature su una **scala comune**
 - evita che la discesa del gradiente risenta eccessivamente della diversità degli ordini di grandezza

Modello

Logistic Regression:

1. Calcola una combinazione lineare delle feature:

$$z = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$$

2. trasforma z in una probabilità tramite la Sigmoid:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

3. applica una soglia (es. 0.5) per ottenere la classe predetta.

Modello

Logistic Regression:

- **interpretabile**: non fornisce solo un'etichetta, ma stima una probabilità;
- **semplice**: permette di evidenziare il confronto tra i metodi di ottimizzazione;
- richiede una rappresentazione numerica delle feature in input
 - le feature del dataset sono già numeriche, per cui non sono necessarie codifiche (es. OHE)

Loss function

Binary Cross-Entropy:

$$L_i(W) = -[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

- y_i : etichetta reale
- $\hat{y}_i$: $p(y_i = 1 \mid x_i)$ predetta
- $(1 - \hat{y}_i)$: $p(y_i = 0 \mid x_i)$

Loss function

La BCE non misura solo l'errore commesso, ma anche la **confidenza** del modello sulla predizione. Ad esempio, se $y_i = 1$:

$$L_i(W) = -\log(\hat{y}_i)$$

- se $\hat{y}_i \approx 1$: il modello è molto «sicuro» che la classe sia 1, per cui la penalizzazione è minima ($L_i \rightarrow 0$);
- se $\hat{y}_i \approx 0$, il modello è molto «sicuro» che la classe sia 0, per cui viene fortemente penalizzato ($L_i \rightarrow +\infty$).

Loss function

Minimizzare la BCE equivale a trovare i parametri W che permettono al modello di effettuare le predizioni probabilistiche il più possibile coerenti con i campioni osservati:

- se $y_i = 1$, vogliamo che $\hat{y}_i \approx 1$;
- se $y_i = 0$, vogliamo che $\hat{y}_i \approx 0$.

Loss function

- Il problema di classificazione binaria viene formulato come un problema di ottimizzazione: trovare i parametri W che minimizzano la BCE (funzione obiettivo);
- i metodi Gradient Descent, SGD e Newton cercano di risolvere questo problema attraverso strategie diverse.

Gradient Descent

```
for epoch in range(1, epochs+1):  
    grad = calcola_gradiente(X_train, y_train, W)  
    W = W - alpha * grad  
    current_loss = calcola_loss(X_train, y_train, W)
```

... e aggiorna i pesi una volta sola.

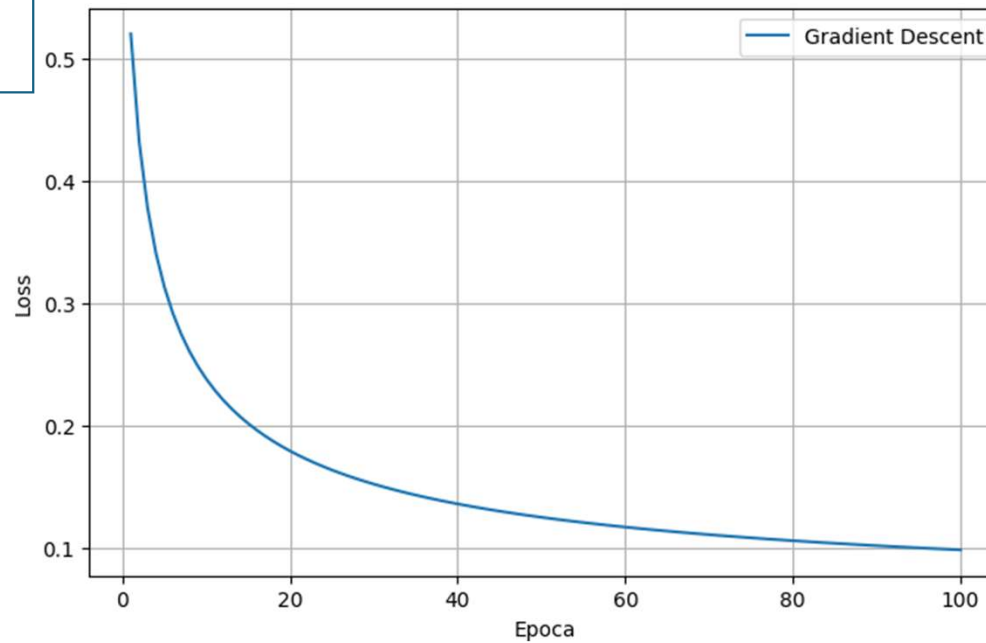
Calcola il gradiente **sull'intero training set...**

Gradient Descent

Comunemente adottato come
valore di partenza nei metodi di
discesa del gradiente

- $\alpha = 0.1$, 100 epoche:

In questo problema
garantisce una
convergenza stabile



Stochastic Gradient Descent

- Caso «estremo» (*1 esempio*):

```
for epoch in range(1, epochs+1):  
    indices = np.random.permutation(N)  
    for i in indices:  
        xi = X_train[i:i+1]  
        yi = y_train[i:i+1]  
        grad = calcola_gradiente(xi, yi, W)  
        W = W - alpha * grad  
    current_loss = calcola_loss(X_train, y_train, W)
```

Shuffle degli esempi

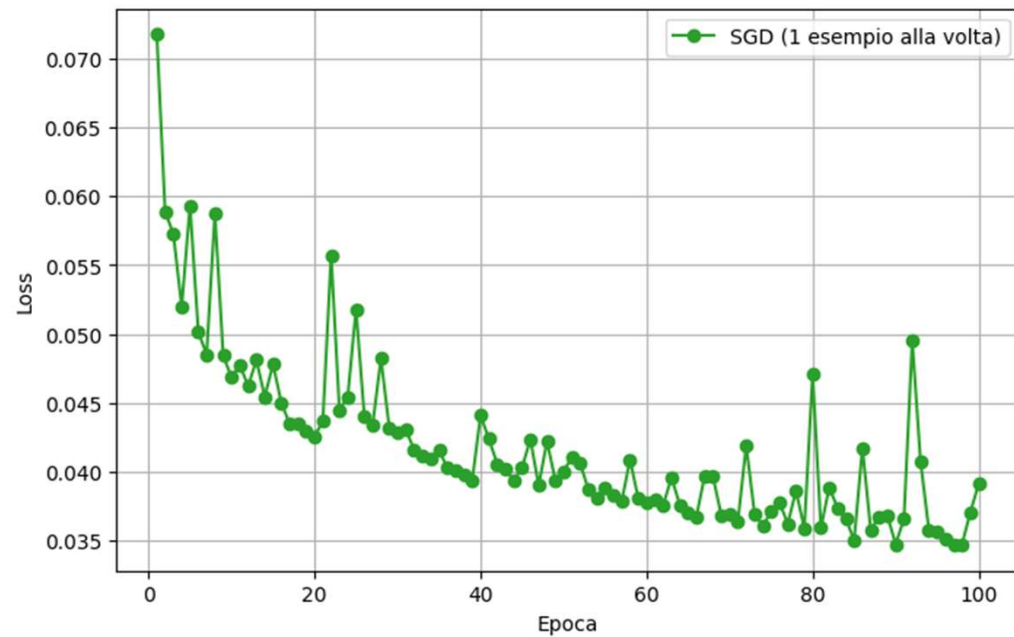
Per ogni esempio...

...e aggiorna i pesi

...calcola il gradiente su quell'esempio...

Stochastic Gradient Descent

- $\alpha = 0.1$, 100 epoche:



Stochastic Gradient Descent

- Caso *mini-batch*:

```
for epoch in range(1, epochs+1):
```

Shuffle degli esempi

```
    indices = np.random.permutation(N)
    X_shuffled = X_train[indices]
    y_shuffled = y_train[indices]
```

Per ogni batch...

```
    for i in range(0, N, batch_size):
```

```
        xb = X_shuffled[i:i+batch_size]
        yb = y_shuffled[i:i+batch_size]
```

...calcola il gradiente su
quel batch...

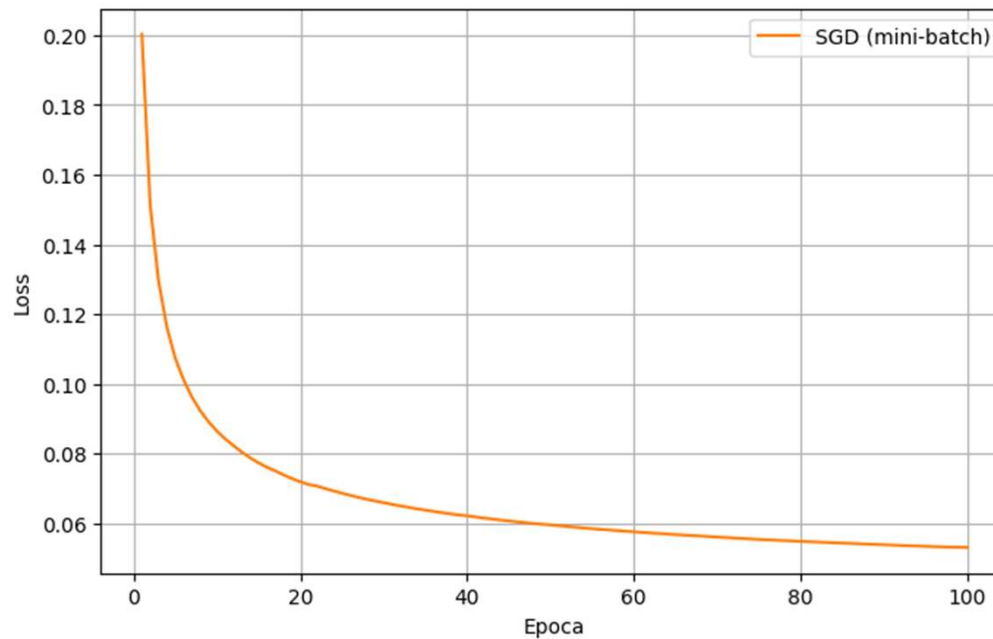
...e aggiorna i pesi

```
        grad = calcola_gradiente(xb, yb, W)
        W = W - alpha * grad
```

```
    current_loss = calcola_loss(X_train, y_train, W)
```

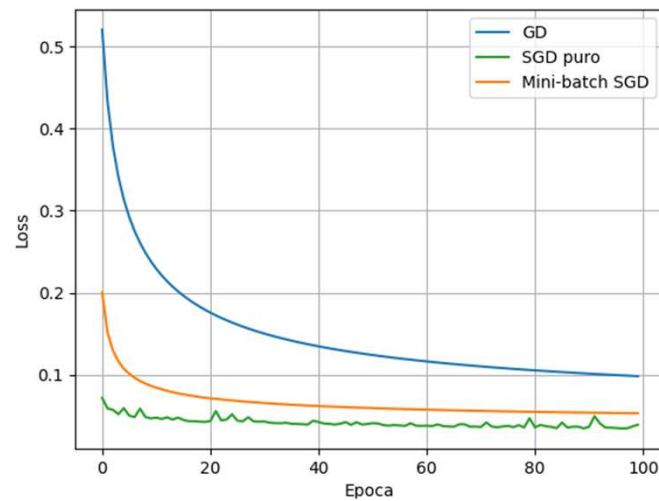

Stochastic Gradient Descent

- $\alpha = 0.1$, 100 epoche:



Confronto

Metodo	Epoche	Loss finale	Tempo (s)	Osservazioni
Gradient Descent	100	0.098281	0.0106	Convergenza stabile, ma loss più elevata
SGD (1 esempio)	100	0.039103	0.5763	Loss minore, ma convergenza non monotona (rumore)
SGD (mini-batch)	100	0.053149	0.0738	Miglior compromesso



Metodo di Newton

```
for i in range(1,iterazioni+1):  
    grad = calcola_gradiente(X_train,y_train,W)  
    H = calcola_hessiana(X_train,y_train,W)  
    p = np.linalg.solve(H,-grad)  
    W = W + p  
    current_loss = calcola_loss(X_train,y_train,W)
```

...calcola l'**Hessiana** per ottenere informazioni sulla **curvatura locale**...

...e aggiorna i pesi
($\alpha = 1$)

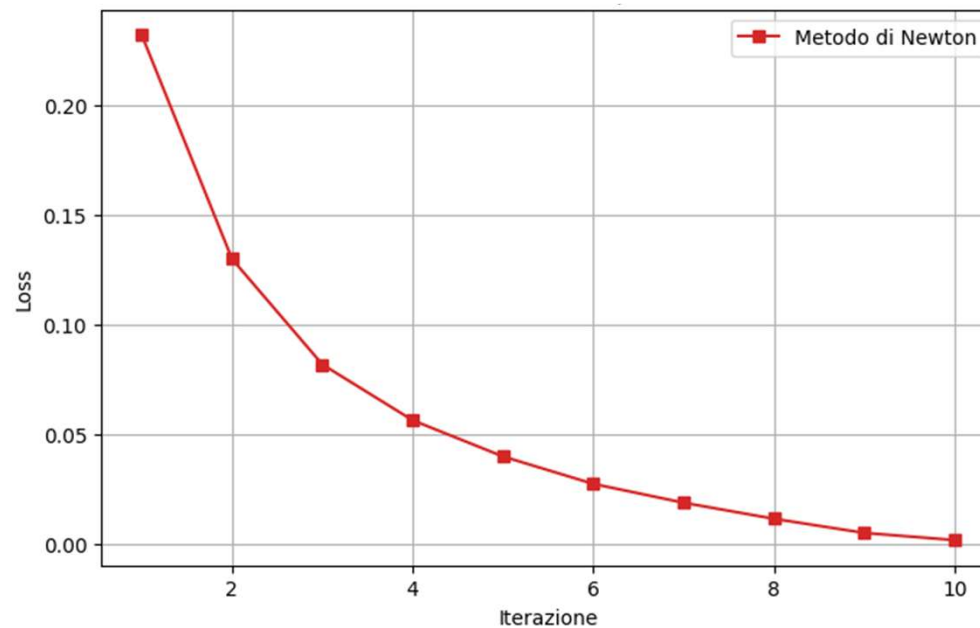
Calcola il gradiente...

...calcola la direzione di discesa sfruttando **gradiente ed Hessiana**...

- Computazionalmente più costoso (informazioni del 2° ordine)
- Converge rapidamente se il punto di partenza x_0 è vicino alla soluzione

Metodo di Newton

- 10 iterazioni:



Metodo di Newton

- In sole 10 iterazioni, il Metodo di Newton raggiunge una BCE sul training pari a 0.001983.
- Tuttavia, il modello mostra una perdita di capacità di generalizzazione: la BCE sul test cresce, nonostante le buone prestazioni di classificazione.

	Loss	Accuracy	Precision	Recall	F1-Score	Matrice di confusione
Training	0.001983	1.0000	1.0000	1.0000	1.0000	$\begin{bmatrix} 170 & 0 \\ 0 & 285 \end{bmatrix}$
Test	1.442194	0.9123	0.9697	0.8889	0.9275	$\begin{bmatrix} 40 & 2 \\ 8 & 64 \end{bmatrix}$

L'aumento della BCE sul test è dovuto a predizioni estremamente confidenti su alcuni campioni classificati erroneamente

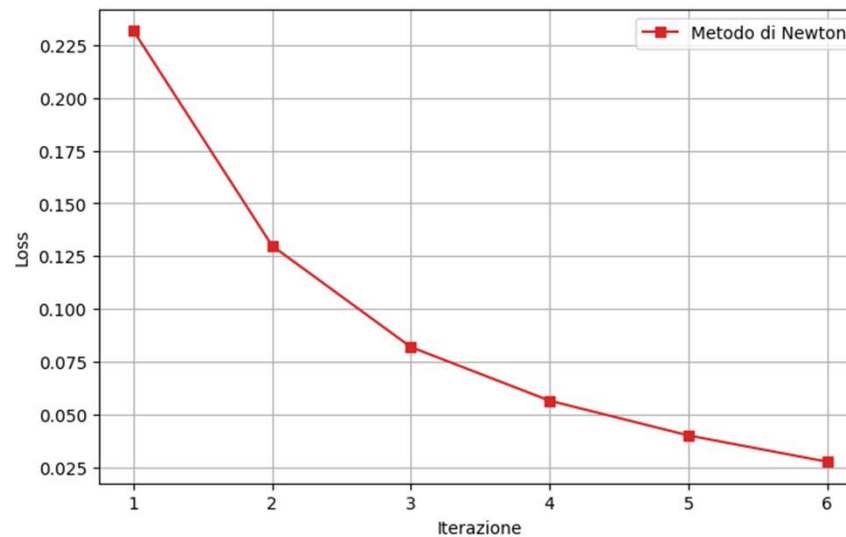
Metodo di Newton

- Riducendo il numero di iterazioni (ad esempio a 6), si ottiene un miglior compromesso tra minimizzazione della loss sul training e capacità di generalizzazione.

	Loss	Accuracy	Precision	Recall	F1-Score	Matrice di confusione
Training	0.027557	0.9912	0.9895	0.9965	0.9930	$\begin{bmatrix} 167 & 3 \\ 1 & 284 \end{bmatrix}$
Test	0.216318	0.9298	0.9706	0.9167	0.9429	$\begin{bmatrix} 40 & 2 \\ 6 & 66 \end{bmatrix}$

Considerazioni finali

- I risultati sperimentali evidenziano la rapida convergenza del metodo di Newton, come mostrato dalla rapida diminuzione della BCE nelle prime iterazioni:



Considerazioni finali

- Nonostante il metodo di Newton richieda un numero ridotto di iterazioni per convergere, ogni iterazione è computazionalmente più costosa rispetto ai metodi precedenti, perché richiede il calcolo dell'Hessiana H e di $p = -H^{-1}\nabla L$.
- In conclusione, la scelta del metodo di ottimizzazione dipende dal **compromesso** tra **velocità di convergenza** e **costo computazionale** di ogni iterazione.

Grazie per l'attenzione.